01

REST API & RETROFIT

dengan Jetpack Compose

Membangun Aplikasi Berita Online

REST API

Retrofit

Jetpack Compose

Kotlin

Coroutines

02

TUJUAN PEMBELAJARAN

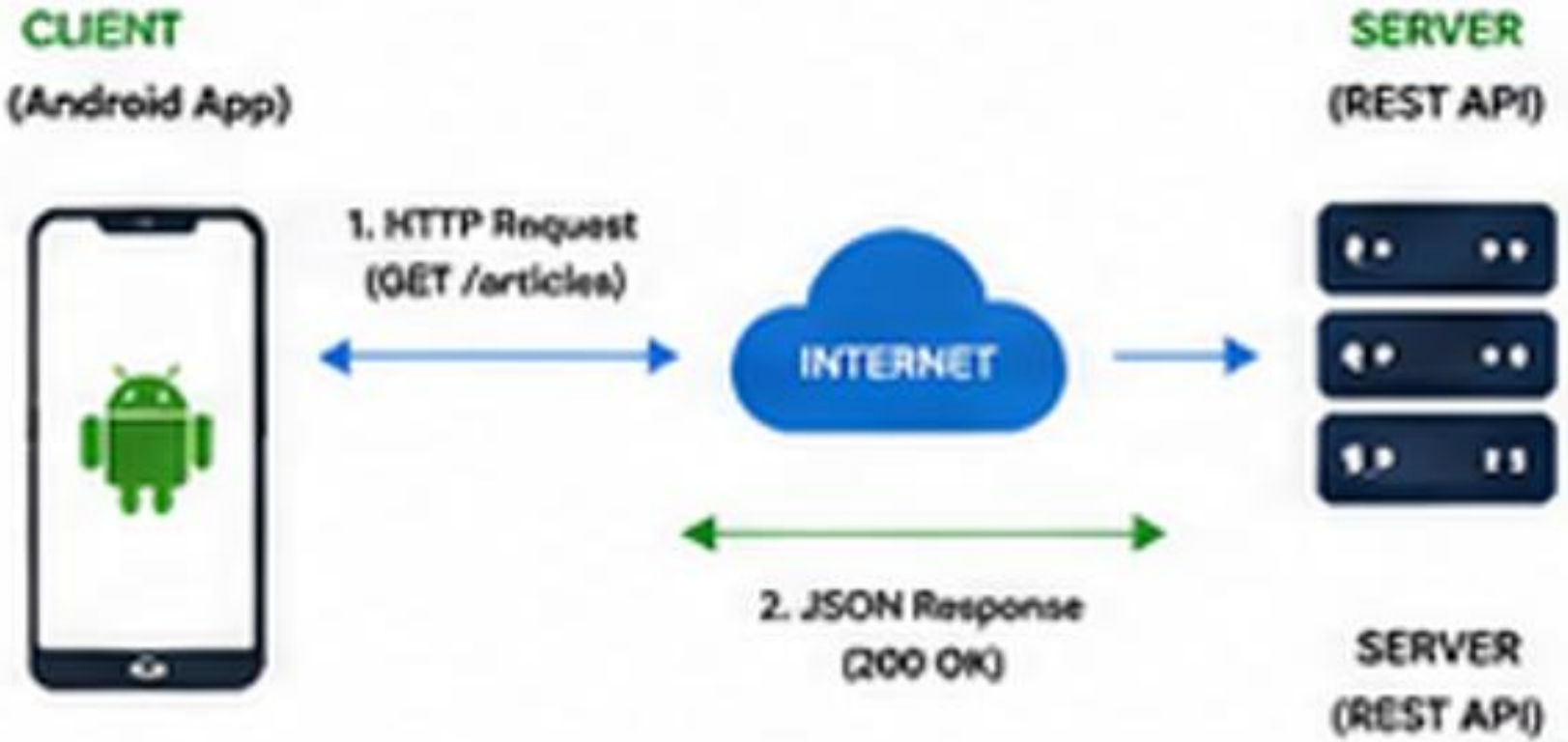
- Memahami konsep REST API
- Memahami format pertukaran data JSON
- Menggunakan Retrofit untuk komunikasi API
- Melakukan GET Request
- Parsing JSON menjadi Object Kotlin
- Menampilkan data API dengan Jetpack Compose
- Membuat aplikasi berita online sederhana

03

APA ITU REST API?

REST (Representational State Transfer) adalah arsitektur komunikasi antara client dan server menggunakan protokol HTTP.

04 CARA KERJA REST API




05
HTTP METHOD
GET
Mengambil data
GET /articles
POST
Menambah data
POST /articles
PUT
Mengubah data
PUT /articles/1
DELETE
Menghpaus data
DELETE /articles/1

06

APA ITU JSON?

JSON (JavaScript Object Notation) adalah format pertukaran data yang ringan, mudah dibaca manusia dan mudah diproses komputer.

```
{
    "id": 1,
    "title": "Belajar Retrofit",
    "author": "Fajar",
    "content": "Isi dari berita...",
    "urlToImage": "https://...",
    "publishedAt": "2024-05-28T10:00:00Z"
}
```

07

STRUKTUR JSON

OBJECT

```
{
   "name": "Budi",
   "age": 20
}
```

ARRAY

```
[
   { "name": "Budi" },
   { "name": "Ani" },
   { "name": "Cici" }
]
```

08

CONTOH JSON BERITA

```
{
   "status": "ok",
   "totalResults": 3,
   "articles": [
     {
        "title": "AI Semakin Populer",
        "author": "Admin",
        "content": "Ini berita...",
        "url": "https://...",
        "urlToImage": "https://...",
        "publishedAt": "2024-05-20T10:00:00Z"
     }
]
```

09

APA ITU RETROFIT?

Retrofit adalah library HTTP client untuk Android dan Java/Kotlin.

- Mudah melakukan request ke REST API
- Otomatis parsing JSON
- Mendukung Coroutines
- Mendukung RxJava
- Mudah diintegrasikan
- Performa tinggi

Retrofit

by square


10
KEUNTUNGAN RETROFIT
SEBELUM (HttpURLConnection)
Kode panjang
Sulit dirawat
Parsing manual
Error handling rumit
DENGAN RETROFIT
Kode lebih sederhana
Parsing otomatis
Mudah digunakan
Modern & fleksibel

11 DEPENDENCY RETROFIT

Tambahkan pada file build.gradle (Module: app)

```
dependencies {
    implementation("com.squareup.retrofit2:retrofit:2.11.0")
    implementation("com.squareup.retrofit2:converter-gson:2.11.0")
    implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")
}
```

12 MEMBUAT API SERVICE

```
interface NewsApiService {

    @GET("top-headlines")
    suspend fun getTopHeadlines(
        @Query("country") country: String = "id",
        @Query("apiKey") apiKey: String
    ) : NewsResponse

    @GET("articles/{id}")
    suspend fun getArticleDetail(
        @Path("id") id: String,
        @Query("apiKey") apiKey: String
    ): Article
}
```

13 MEMBUAT RETROFIT INSTANCE

```
object RetrofitInstance {
    private const val BASE_URL = "https://newsapi.org/v2/"
    val api: NewsApiService by lazy {
        Retrofit.Builder()
            .addConverterFactory(GsonConverterFactory.create())
            .addCallAdapterFactory(CoroutineCallAdapterFactory())
            .client(getOkHttpClient())
            .build()
            .create(NewsApiService::class.java)
    }
    private fun getOkHttpClient(): OkHttpClient {
        val logging = HttpLoggingInterceptor()
        logging.level = HttpLoggingInterceptor.Level.BODY
        return OkHttpClient.Builder()
            .addInterceptor(logging)
    }
}
```

14 MENGIRIM GET REQUEST

```
class NewsRepository(
    private val api: NewsApiService
)
    suspend fun getTopHeadlines(): NewsResponse {
        return api.getTopHeadlines(apiKey = BuildConfig.API_KEY)
    }
    suspend fun getArticleDetail(id: String): Article {
        return api.getArticleDetail(id, apiKey = BuildConfig.API_KEY)
}
```

15 DATA CLASS (MODEL)

JSON

```
{
  "title": "AI News",
  "author": "Admin",
  "url": "https://...",
  "urlToImage": "https://..."
}
```

→

Kotlin Data Class

```
data class Article(
    val title: String,
    val author: String?,
    val url: String,
    val urlToImage: String?
)
```

16 PARSING OTOMATIS

JSON
(Response)

(Converter Factory)

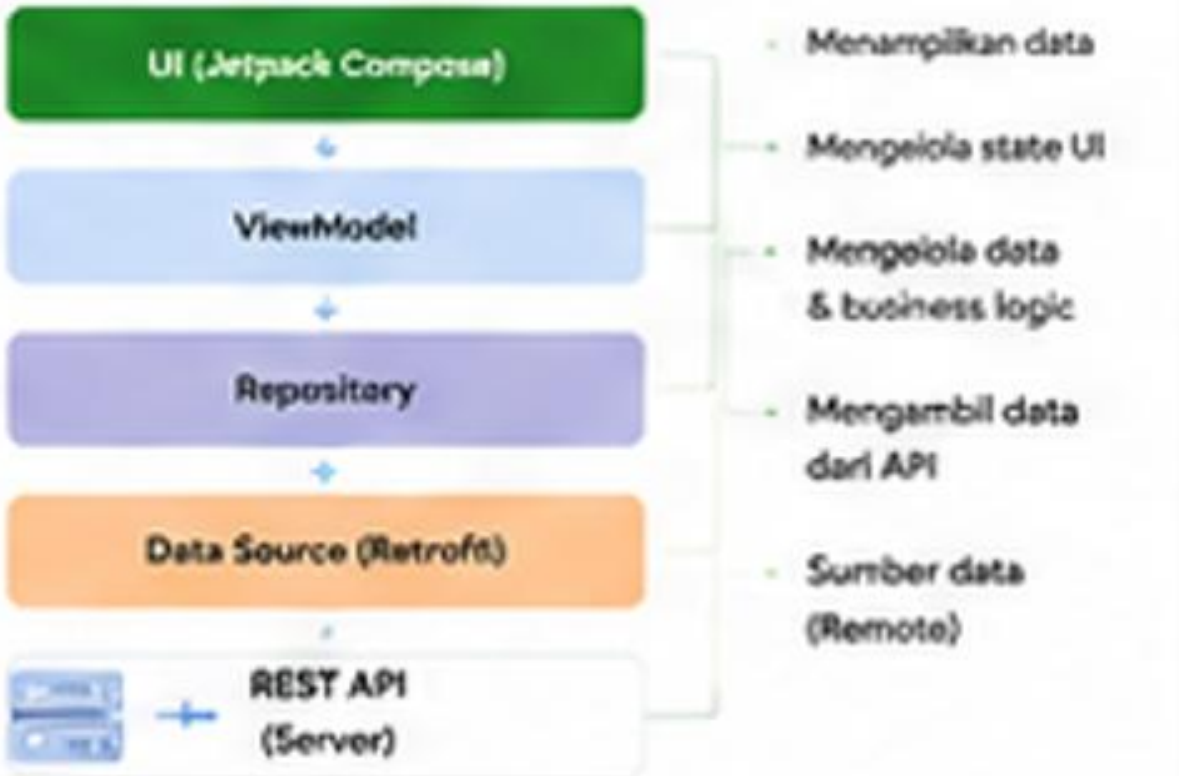
Object Kotlin
(Data Class)

Retrofit + Gson = Auto Parsing
(JSON → Object Kotlin)

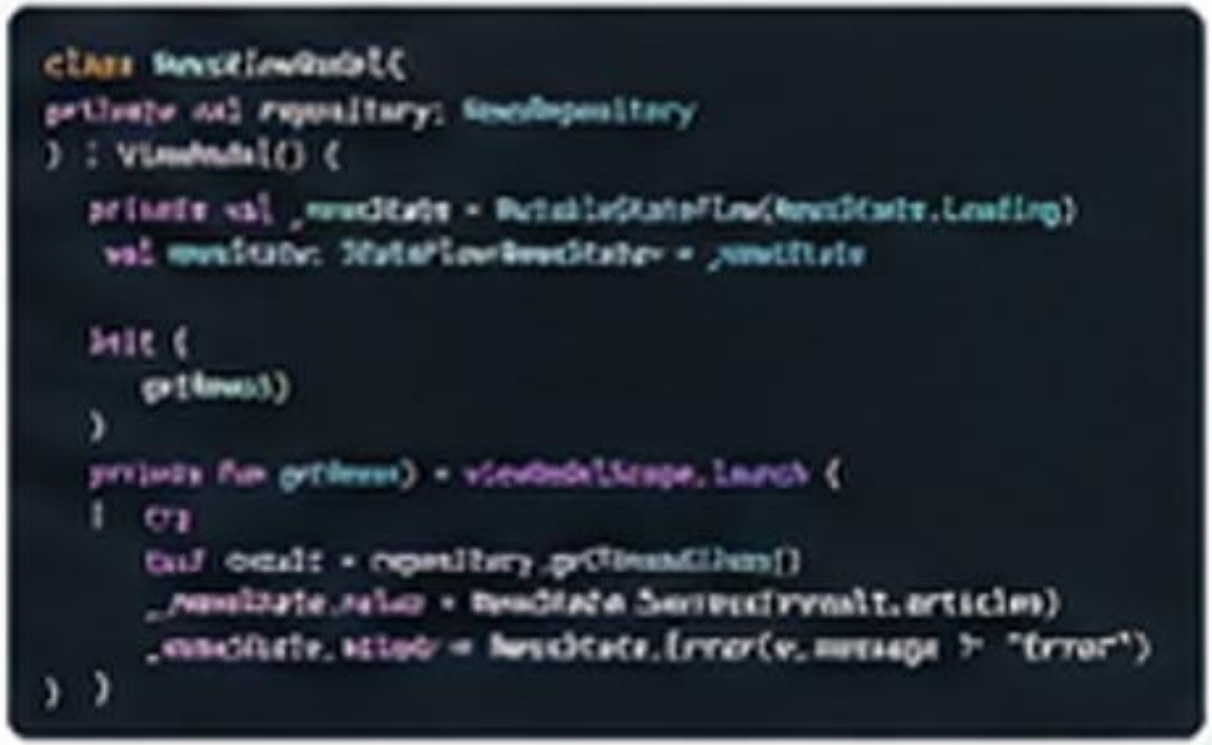
17 RESPONSE MODEL

```
data class NewsResponse(
    val status: String,
    val totalResults: Int,
    val articles: List<Article>
)
```

13 ARCHITECTURE (MVVM)



19 VIEWMODEL



20 UI STATE

```
sealed class NewsState {
    object Loading : NewsState()
    data class Success(val data: List<Article>) : NewsState()
    data class Error(val message: String) : NewsState()
}

// Penggunaan di Compose
when (state) {
    is NewsState.Loading -> LoadingView()
    is NewsState.Success -> NewsListView(state.data)
    is NewsState.Error -> ErrorView(state.message)
}
```

21 MENAMPILKAN LIST BERITA

```
@Composable
fun NewsList(articles: List<Article>, onClick) -> Unit) {
  LazyColumn(contentPadding = PaddingValues(0.dp)) {
    items(articles) { article ->
    NewsItem(article = article, onClick = { onClick(article) })
    Divider()
  }
}
```

22 ITEM BERITA (COMPOSE)

```
@Composable
fun NewsItem(article: Article, onClick: () -> Unit) {
    Card(modifier = Modifier.fillMaxWidth().clickable(onClick = onClick)) {
        Row(Modifier.padding(12.dp), verticalAlignment = Alignment.CenterVertically) {
        AsyncImage(model = article.urlToImage, contentDescription = null,
         modifier = Modifier.size(80.dp).clip(RoundedCornerShape(8.dp)),
         contentScale = ContentScale.Crop)
         Spacer(modifier = Modifier.width(12.dp))
         Column(modifier.weight(1f)) {
         Text(article.title, fontWeight = FontWeight.Bold, maxLines = 2, overflow = TextOverflow.Ell
         Text(article.author.orEmpty(), style = MaterialTheme.typography.bodySmall)
    } }
}
```

23 DETAIL BERITA (COMPOSE)

```
@Composable
fun ArticleDetail(article: Article) {
    Column(Modifier.verticalScroll(rememberScrollState()).padding(16.dp)) {
    AsyncImage(model = article.urlToImage, contentDescription = null,
        Modifier = Modifier.fillMaxWidth().height(200.dp).clip(RoundedCornerShape(8.dp)),
        contentScale = ContentScale.Crop)
    }
    Spacer(Modifier.height(16.dp))
    Text(article.title, style = MaterialTheme.typography.headlineSmall,
      fontWeight = FontWeight.Bold)
      Text = MaterialTheme.colorScheme.onSurface);
    Spacer(Modifier.height(16.dp))
}   Text(article.content ?: "Konten tidak tersedia.")
```

23 NAVIGASI DENGAN NAVIGATION COMPOSE

```
@Composable
fun NewsNavGraph() {
    val navController = rememberNavController()
    NavHost(navController, startDestination = "home") {
        composable("home") {
        HomeScreen(onItemClick = { id -> navController.navigate("detail/$id") })
    }
    composable("detail/{id}", arguments = listOf(navArgument("id") { type = NavType.StringType })
    ) { backStackEntry ->
        val id = backStackEntry.arguments?.getString("id") ?: ""
        DetailScreen(id)
    }
}
```

25

ALUR APLIKASI

App Start → Load API → Fetch JSON → Parsing (JSON → Object) → List Artikel → Detail Berita

26

STUDI KASUS: APLIKASI BERITA ONLINE

Kita akan membangun aplikasi berita online dengan fitur:

- Fetch data berita dari API
- Menampilkan list artikel
- Menampilkan detail berita
- Pull To Refresh
- Search berita
- Error Handling

27

DESAIN UI HOME SCREEN

Toolbar

Search

List Artikel

Author & Date

28

DESAIN UI DETAIL SCREEN

Gambar Artikel

Judul

Author & Date

Konten Berita

Link Sumber

29

FITUR TAMBAHAN

Search News

Pull To Refresh

Loading Indicator

Error Handling

Share Article

Open In Browser

30

KESIMPULAN

- REST API adalah cara aplikasi berkomunikasi dengan server
- JSON adalah format pertukaran data yang ringan
- Retrofit memudahkan HTTP request & parsing JSON
- Jetpack Compose menampilkan data secara modern & deklaratif
- MVVM + Coroutines membuat arsitektur aplikasi bersih & efisien

SELAMAT KODING! 🚀